Python at the roofline: nine GPU workloads against hand-written CUDA and established codes

Walid Hemati
Independent researcher

Abstract

We test one claim across six simulation codes: GPU kernels written and launched from Python reach the same *hardware efficiency* as hand-written CUDA and as established high-performance codes. The six open-source packages cover lattice-Boltzmann flow (D2Q9 and D3Q19), explicit stencils (2D and 3D), direct-summation N-body gravity, Lennard-Jones molecular dynamics, pseudo-spectral Navier–Stokes (2D and 3D) and Ising Monte Carlo, nine workloads in all. Each is implemented along the same ladder (NumPy, PyTorch, `torch.compile`, the Python GPU languages Triton, NVIDIA Warp and Numba-CUDA, and hand-written CUDA C compiled at run time), validated against analytic or reference results, and measured on one NVIDIA GeForce RTX 3070 Laptop GPU against a ceiling measured on that GPU: copy bandwidth (361 GB/s) for memory-bound kernels, FP32 FMA throughput (15.5 TFLOPS) for N-body, cuFFT for spectral methods and copy bandwidth plus Philox throughput for Ising. Across the nine workloads the best kernel written entirely in a Python GPU language reaches 77–103 % (median 97 %) of the efficiency of our hand-written CUDA C. We also ran established codes on the same GPU: FluidX3D, Lettuce, pystencils, AN5D, the CUDA Samples `nbody`, LAMMPS, HOOMD-blue, JAX-CFD, spectralDNS, Dedalus and NVIDIA's `ising-gpu`. At the levels compared, the Python-launched kernels match or exceed the best of them on all nine workloads; the exception is the temporal-blocking rung of the 2D stencil, where the code generator AN5D is ahead. Where Python falls short of CUDA C, the missing piece is a schedule (a marching order, a fused transform plan), not the language.

1 Introduction

Scientific GPU codes are usually written in CUDA C/C++ or Fortran, and Python is used to drive them. Python-hosted GPU languages (Triton [1], NVIDIA Warp [2], Numba-CUDA [3]) and compilers for array code (`torch.compile` [4]) now let the kernel itself be written in Python. Whether such kernels are competitive is often argued with single benchmarks on different hardware. We instead measure every kernel as a fraction of a roofline [5] measured on the same GPU, for nine workloads from six independent packages, and we run established codes for the same workloads on that GPU.

Contributions:

- Six open-source packages (`pylbm`, `pyheat3d`, `pygravity`, `pymd`, `pyspectral`, `pyising`), each with the same backend ladder, a NumPy oracle, physics validation and a benchmark that writes every number of its paper from a results file.
- A common measurement protocol: one GPU, measured ceilings, minimal traffic models, serial runs on an idle GPU, medians of five repeats.

- Same-GPU head-to-heads with eleven established codes, with each counterpart checked to solve the same problem as ours.
- A cross-workload comparison (fig. 1 and table 1) of where Python GPU languages reach hand-written CUDA C, where `torch.compile` suffices, and where neither does.

Each package has its own paper with the full method, validation and results; this paper summarises them and draws the common conclusions.

2 Method

Hardware and ceilings. All measurements use one NVIDIA GeForce RTX 3070 Laptop GPU (GA104, 40 SMs, 8 GB GDDR6, 448 GB/s specification bandwidth). The device-to-device copy bandwidth measured with 1 GiB buffers is 361 GB/s; an FMA-chain microbenchmark gives 15.5 TFLOPS in FP32; the Philox4x32-10 generator [6] used for Monte Carlo runs at 112.6 calls per ns; cuFFT [7] times are measured per transform size.

Rooflines. Each workload uses the ceiling that bounds it (table 1). For memory-bound kernels the bound is copy bandwidth divided by the *minimal* DRAM traffic of one update (e.g. $2Q$ populations for lattice Boltzmann, one read and one write per stencil point, read-own/read-neighbour/write-own per spin). The minimal model never inflates effective bandwidth; kernels slightly above 100 % move a little less than the copy kernel's traffic per byte, or hit in the L2 cache. For N-body the bound is the FMA peak at 20 flops per interaction. For pseudo-spectral DNS it is the time of the step's FFTs as standalone cuFFT calls plus the minimal traffic of the pointwise work. For Ising it is the smaller of the bandwidth bound and the Philox bound, with the random numbers a spin update actually needs.

The ladder. Every package implements the same algorithm in: NumPy (the correctness oracle), PyTorch eager [8], `torch.compile` (the same tensor code, compiled by Inductor into Triton), hand-written kernels in Triton, Warp and Numba-CUDA ("Python GPU languages" below), and hand-written CUDA C compiled at run time by NVRTC through CuPy [9] and launched from Python. All backends share one interface and are checked against the NumPy oracle; most packages reproduce it bit for bit.

Measurement. Benchmarks warm up (including JIT compilation and autotuning), synchronise the device around timed regions, run each timed repeat for at least 0.5 s and report the median of five repeats. The final numbers come from one serial run per package on an otherwise idle GPU. For each implementation class we report the best variant at the largest problem size it reached: small grids can exceed the DRAM roofline because part of the working set stays in the 4 MB L2 cache.

Counterparts. For each workload we chose established codes that solve the same problem and could be built here, built them from pinned commits without changing their algorithms, and checked that they reproduce our results before timing them. Where the counterpart runner applied our traffic model or ceiling directly (lattice Boltzmann, stencils, N-body) we quote that percentage. Otherwise (MD, spectral, Ising) we divide the counterpart's throughput by our CUDA C throughput measured in the same session at the same problem size, and multiply by our kernel's roofline fraction. The details, including every build patch, are in the package papers.

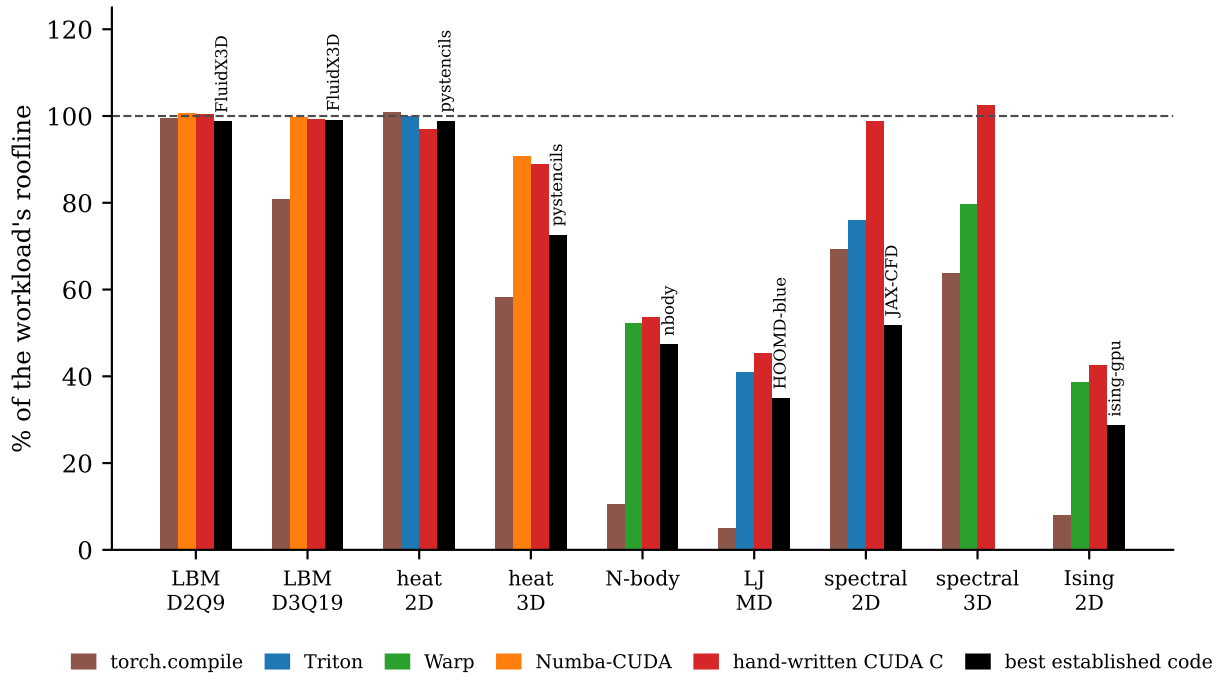


Figure 1: Best implementation of each class as a percentage of the workload's roofline: `torch.compile`, the best kernel written in a Python GPU language (bar colour: Triton, Warp or Numba), hand-written CUDA C launched from Python, and the best established code run on the same GPU. Dashed: the roofline.

3 Results

Figure 1 and table 1 summarise the nine workloads.

Lattice Boltzmann. Fused stream–collide kernels in all three Python languages run at the copy bandwidth: 100 % (D2Q9) and 100 % (D3Q19), the same as CUDA C (100 %, 99 %) and as the hand-optimised OpenCL code FluidX3D [10] (99 %, 99 %). `torch.compile` reaches the roofline in 2D (100 %) but not in 3D (81 %). The PyTorch framework Lettuce [11] with its native kernels is far below.

Stencils. Single-step heat-equation kernels reach 100 % (2D) and 91 % (3D) in a Python language, against 97 % and 89 % for CUDA C and 99 % and 73 % for pystencils [12]. In 3D the difference is the schedule: kernels that march along z reach the bound, one thread per point (pystencils, `torch.compile` at 58 %) does not. Temporal blocking raises the per-update roofline: our CUDA C kernels reach 3.62× the single-step roofline in 2D ($k = 8$ fused steps) and 2.09× in 3D ($k = 3$); AN5D [13] reaches 5.24× ($k = 13$) and 1.59× ($k = 4$). This rung is CUDA C, because temporal blocking needs explicit shared-memory schedules that Triton does not expose.

***N*-body.** The tiled force kernel reaches 52 % of the FMA peak in Warp and 54 % in CUDA C; the CUDA Samples `nbody` [14, 15] reaches 47 % with fast math. Instruction counts explain the level: an interaction needs more issue slots than its 20 nominal flops (see the `pygravity` paper). `torch.compile` reaches 11 %.

Table 1: Percent of each workload's roofline at the largest problem size (fig. 1). The last column lists the best one or two established codes.

workload	bound	compile	Python GPU lang.	CUDA C	established code
LBM D2Q9	memory	100	100 (Numba-CUDA)	100	FluidX3D 99; Lettuce 27
LBM D3Q19	memory	81	100 (Numba-CUDA)	99	FluidX3D 99; Lettuce 14
heat 2D	memory	101	100 (Triton)	97	pystencils 99
heat 3D	memory	58	91 (Numba-CUDA)	89	pystencils 73
N-body	compute	11	52 (Warp)	54	CUDA Samples nbody 47
LJ MD	memory	5	41 (Triton)	45	HOOMD-blue 35; LAMMPS 12
spectral 2D	FFT	69	76 (Triton)	99	JAX-CFD 52
spectral 3D	FFT	64	80 (Warp)	102	–
Ising 2D	memory/RNG	8	38 (Warp)	42	NVIDIA ising-gpu 29

Molecular dynamics. With neighbour lists, the best Python-language kernel (Triton) reaches 41 % of the bandwidth bound and CUDA C 45 %, against 35 % for HOOMD-blue [16] and less for LAMMPS [17] with its KOKKOS [18] and GPU packages. The traffic model counts only the minimal neighbour-list and position traffic, so these fractions are lower bounds.

Pseudo-spectral DNS. The fused in-place CUDA C kernels around batched cuFFT plans reach the FFT roofline (99 % in 2D, 102 % in 3D). The Python languages reach 76 % and 80 %, `torch.compile` 69 % and 64 %. JAX-CFD [19] reaches 52 % of the same roofline in 2D; spectralDNS [20] (CPU) and the GPU branch of Dedalus [21] are compared in the `pyspectral` paper.

Ising Monte Carlo. One-bit multi-spin coding reaches 38 % of the roofline in Warp and 42 % in CUDA C; NVIDIA's `cuIsingModel` [22, 23] reaches 29 % and the 4-bit code of Romero et al. [24] less. Neither code reaches the bandwidth bound; the work per spin, mostly random-number generation, sets the level.

4 Discussion

Python GPU languages reach CUDA C. On every workload the best Triton, Warp or Numba kernel is within a few percent of our CUDA C, or ahead, except pseudo-spectral DNS (77 % and 78 % of the CUDA C efficiency), molecular dynamics (90 %) and Ising (91 %). In the spectral codes the gap is in the transforms: the CUDA C path uses in-place batched cuFFT plans and fuses the pointwise work around them, while the Python-language kernels call the library through higher-level interfaces. In MD and Ising the kernels implement the same algorithm; the package papers attribute the remaining differences to code generation (vector versus scalar loads, 64-bit index arithmetic, unrolling), measured instruction by instruction for N-body and not profiled in depth for the others.

`torch.compile` is enough for one fused loop. When the whole step is a pointwise expression over a regular grid (2D lattice Boltzmann, the 2D stencil), `torch.compile` reaches the roofline with no kernel code at all. It falls short where the efficient kernel needs a schedule that the tensor program does not express: marching through z in 3D, shared-memory tiling for N-body, neighbour-list gathers in MD, bit-sliced logic for Ising.

Established codes. On this GPU the Python-launched kernels match or exceed the best established code on all nine workloads of fig. 1, and lose on one further rung, deep temporal blocking in 2D. Several counterparts were tuned for other GPUs (AN5D for P100/V100, `ising-gpu` on data-centre parts) and some run their own test case (FluidX3D, Lettuce); the package papers state each difference. The one clear loss, AN5D's deep 2D temporal blocking, is a comparison between hand-written and generated CUDA, not between Python and C. We read the head-to-heads as evidence that the Python-launched kernels are not slow reference implementations, not as rankings of those codes.

Threats to validity. (i) One laptop GPU: clocks depend on temperature and power, and absolute rates do not transfer; percentages of measured ceilings transfer better. (ii) Our CUDA C is compiled by NVRTC, not `nvcc` with tuned flags. (iii) Each roofline is a model; the minimal traffic models make memory-bound fractions upper bounds on efficiency for the kernels and lower bounds on the bandwidth they achieve. (iv) Counterparts were built from pinned commits with minimal build patches and not retuned for this GPU. (v) The "best variant" of a class is chosen after the fact from the measured runs.

5 Conclusion

Across nine GPU workloads, kernels written in Python GPU languages reach 77–103 % of the efficiency of hand-written CUDA C on the same GPU, and the best Python-launched kernels match or exceed the best established code on each of them, with deep 2D temporal blocking as the one rung where a generated CUDA code is ahead. What decided the result was always the algorithm and its schedule: minimal traffic, fused passes, tiling, random bits per spin, FFT counts. Where those are expressed, Python reaches the roofline.

Code availability

All code is MIT-licensed at `https://github.com/walidhemati`: the repositories `pylbm`, `pyheat3d`, `pygravity`, `pymd`, `pyspectral`, `pyising` and, for this summary, `pyhpc-overview`. `collect.py` and `figures/make_figures.py` regenerate every number here from the packages' results files.

References

[1] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL 2019)*, pages 10–19, 2019. doi: 10.1145/3315508.3329973.

[2] Miles Macklin. Warp: A high-performance python framework for GPU simulation and graphics. NVIDIA GPU Technology Conference (GTC), `https://github.com/nvidia/warp`, March 2022.

[3] Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. Numba: A LLVM-based Python JIT compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC (LLVM '15)*, pages 1–6, 2015. doi: 10.1145/2833157.2833162.

[4] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, et al. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*, pages 929–947, 2024. doi: 10.1145/3620665.3640366.

[5] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

[6] John K. Salmon, Mark A. Moraes, Ron O. Dror, and David E. Shaw. Parallel random numbers: As easy as 1, 2, 3. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis (SC '11)*, pages 1–12, New York, NY, USA, 2011. ACM. doi: 10.1145/2063384.2063405.

[7] NVIDIA Corporation. cuFFT library documentation. https://docs.nvidia.com/cuda/cufft/. Version 13.4 documentation at time of access.

[8] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, et al. PyTorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf.

[9] Ryosuke Okuta, Yuya Unno, Daisuke Nishino, Shohei Hido, and Crissman Loomis. CuPy: A NumPy-compatible library for NVIDIA GPU calculations. In *Proceedings of Workshop on Machine Learning Systems (LearningSys) in the Thirty-first Annual Conference on Neural Information Processing Systems (NIPS)*, 2017. URL http://learningsys.org/nips17/assets/papers/paper_16.pdf.

[10] Moritz Lehmann, Mathias J. Krause, Giorgio Amati, Marcello Sega, Jens Harting, and Stephan Gekle. Accuracy and performance of the lattice Boltzmann method with 64-bit, 32-bit, and customized 16-bit number formats. *Physical Review E*, 106(1):015308, 2022. doi: 10.1103/PhysRevE.106.015308.

[11] Mario Christopher Bedrunka, Dominik Wilde, Martin Kliemank, Dirk Reith, Holger Foysi, and Andreas Krämer. Lettuce: PyTorch-based lattice Boltzmann framework. In Heike Jagode, Hartwig Anzt, Hatem Ltaief, and Piotr Luszczek, editors, *High Performance Computing: ISC High Performance Digital 2021 International Workshops, Frankfurt am Main, Germany, June 24 – July 2, 2021, Revised Selected Papers*, volume 12761 of *Lecture Notes in Computer Science*, pages 40–55, Cham, 2021. Springer. doi: 10.1007/978-3-030-90539-2_3.

[12] Martin Bauer, Johannes Hötzer, Dominik Ernst, Julian Hammer, Marco Seiz, Henrik Hierl, Jan Hönig, Harald Köstler, Gerhard Wellein, Britta Nestler, and Ulrich Rüde. Code generation for massively parallel phase-field simulations. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC19)*, pages 1–32, 2019. doi: 10.1145/3295500.3356186.

[13] Kazuaki Matsumura, Hamid Reza Zohouri, Mohamed Wahib, Toshio Endo, and Satoshi Matsuoka. AN5D: Automated stencil framework for high-degree temporal blocking on GPUs. In

Proceedings of the 18th ACM/IEEE International Symposium on Code Generation and Optimization (CGO 2020), pages 199–211, 2020. doi: 10.1145/3368826.3377904.

[14] Lars Nyland, Mark Harris, and Jan Prins. Fast N-body simulation with CUDA. In Hubert Nguyen, editor, *GPU Gems 3*, chapter 31, pages 677–695. Addison-Wesley, 2007.

[15] NVIDIA Corporation. CUDA samples. https://github.com/NVIDIA/cuda-samples, 2026. Sample `cpp/5_Domain_Specific/nbody`, commit 5443602 (v13.4), accessed 2026-10-10.

[16] Joshua A. Anderson, Jens Glaser, and Sharon C. Glotzer. HOOMD-blue: A Python package for high-performance molecular dynamics and hard particle Monte Carlo simulations. *Computational Materials Science*, 173:109363, 2020. doi: 10.1016/j.commatsci.2019.109363.

[17] Aidan P. Thompson, H. Metin Aktulga, Richard Berger, Dan S. Bolintineanu, W. Michael Brown, et al. LAMMPS – a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales. *Computer Physics Communications*, 271:108171, 2022. doi: 10.1016/j.cpc.2021.108171.

[18] Christian R. Trott, Damien Lebrun-Grandié, Daniel Arndt, Jan Ciesko, Vinh Dang, Nathan Ellingwood, et al. Kokkos 3: Programming model extensions for the exascale era. *IEEE Transactions on Parallel and Distributed Systems*, 33(4):805–817, 2022. doi: 10.1109/TPDS.2021.3097283.

[19] Gideon Dresdner, Dmitrii Kochkov, Peter Norgaard, Leonardo Zepeda-Núñez, Jamie A. Smith, Michael P. Brenner, and Stephan Hoyer. Learning to correct spectral methods for simulating turbulent flows, 2022.

[20] Mikael Mortensen and Hans Petter Langtangen. High performance Python for direct numerical simulations of turbulent flows. *Computer Physics Communications*, 203:53–65, 2016. doi: 10.1016/j.cpc.2016.02.005.

[21] Keaton J. Burns, Geoffrey M. Vasil, Jeffrey S. Oishi, Daniel Lecoanet, and Benjamin P. Brown. Dedalus: A flexible framework for numerical simulations with spectral methods. *Physical Review Research*, 2(2):023068, 2020. doi: 10.1103/PhysRevResearch.2.023068.

[22] Mauro Bisson, Massimo Bernaschi, Massimiliano Fatica, Nikolaos G. Fytas, Isidoro González-Adalid Pemartín, Víctor Martín-Mayor, and Alexandros Vasilopoulos. Massive-scale simulations of 2D Ising and Blume-Capel models on rack-scale multi-GPU systems. *Computer Physics Communications*, 315:109690, 2025. doi: 10.1016/j.cpc.2025.109690.

[23] NVIDIA. ising-gpu: GPU-accelerated Monte Carlo simulations of 2D Ising model. https://github.com/NVIDIA/ising-gpu, 2025. Commit 935796d.

[24] Joshua Romero, Mauro Bisson, Massimiliano Fatica, and Massimo Bernaschi. High performance implementations of the 2D Ising model on GPUs. *Computer Physics Communications*, 256:107473, 2020. doi: 10.1016/j.cpc.2020.107473.